Computação Móvel
Mobile Computing
CM

Tutorial 4 - Flows&Firebase

Pedro Fazenda
pedrofazenda@enautica.pt

Carlos Gonçalves
carlosgoncalves@enautica.pt

Abstract

This tutorial introduces students to Kotlin Flows and coroutines for reactive programming, demonstrates how to access AI language models (such as OpenAI's GPTs and Google Gemini) using Kotlin, and provides an overview of the Google Firebase platform.

Deadline: June 1st, 2026

Kotlin Flows, AI LLMs and Firebase

Friday, April 24th, 2026

Contents

1 Introduction

In this tutorial, you will explore Kotlin Fows, access to LLM's API and Google Firebase.

1.1 Assignment Overview

It is about Kotlin Fows, access to LLM's API and Google Firebase.

1.2 Learning Objectives

By completing this assignment, you will have the following experience:

- in threading and asynchronous code (threads, callbacks, coroutines, flows, channels);
- accessing OpenAI GPT and Google Gemini LLMs (you will make calls, regulate calls temperature and other parameters, and do sentiment analysis); and
- with Google Firebase.

2 Kotlin Flows

2.1 Introduction to coroutines/flows

Kotlin Flows are a powerful tool for handling asynchronous data streams in a reactive and declarative way. Introduced as part of Kotlin's coroutines framework, Flows represent a cold asynchronous stream of values that are computed lazily. Unlike traditional callbacks or LiveData, Flows provide a consistent and structured approach to managing asynchronous sequences — making them ideal for modern Android development.
A typical use case for Flows is observing changes in data over time, such as updates from a database, network responses, or user input events. A Flow emits multiple values sequentially, and consumers collect these values using the collect function, which is itself a suspend function —meaning it must be called within a co-routine.
Flows are cold by default, meaning the code inside the flow builder doesn't run until the Flow is collected. This makes them efficient and predictable. Flows can also be transformed using operators like map, filter, combine, and flatMapLatest, allowing developers to build complex data pipelines in a concise and readable manner.
Error handling is built in through operators such as catch and onEach, while Flow cancellation is automatic when the collecting co-routine is canceled, preventing memory leaks.
In Android, Kotlin Flows integrates well with Jetpack libraries like Room and ViewModel. For example, a Room DAO can return a Flow<List<Item>> that automatically updates the UI whenever the database changes. Paired with StateFlow or SharedFlow — which are hot variants of Flow — you can handle state and events more explicitly.
Overall, Kotlin Flows promote a clean and reactive programming model. They replace callback-heavy designs with structured and testable code, making apps more responsive

and easier to maintain. For any Android developer using co-routines, understanding and using Flows is an essential skill.

2.2 Step 1: Kotlin Coroutines/Flows Tutorial

Execute the "Coroutines and channels - tutorial" located in:
https://kotlinlang.org/docs/coroutines-and-channels.html.

[Text extracted from the tutorial]
In this tutorial, you will learn how to use coroutines to perform network requests without blocking the underlying thread or callbacks.
You will learn:

- Why and how to use suspending functions to perform network requests;
- How to send requests concurrently using co-routines;
- How to share information between different co-routines using channels.

For network requests, you will need the Retrofit library, but the approach shown in this tutorial works similarly for any other libraries that support coroutines.
[End of extracted text]

The tutorial guides you through transforming a blocking network request application into a responsive, concurrent one using coroutines and channels, in an application with a java Swing interface. Here is an overview of the code experiments in the tutorial:

- Blocking - using the same thread for UI and requests - freezes UI
- Background - using a background thread - better but resource intensive
- Callbacks - using callback - error-prone
- Suspend - using suspend functions and coroutines
- Concurrent - with concurrent execution
- Not cancelable - understanding coroutine cancellation
- Progress - displaying intermediate results
- Channels - using channels

2.3 Step 2: DIY, a more reactive handling of loading states

Start by making a copy of the project, name the directory intro-coroutinesV2, and execute the following tasks on that copy.

Change the code to use StateFlow for better management of load status

Instead of manually updating the loading status, use a StateFlow to represent and observe the loading state.
Therefore, each time we change loadingStatus, a flow item is generated in a StateFlow asynchronous flow, and a collector is running to collect those items and send them to the UI.
This will make state management more reactive and provide better encapsulation of the loading status.
Steps to be taken:

1. Complement the LoadingStatus enumerate with a LoadingStateData data class
2. Add a StateFlow property to the Contributors interface and a mutable state flow Backing Property Pattern in ContributorUI class
3. Update the methods that manage loading status to use StateFlow

2.3.1 Task 1: Add a LoadingStateData data class

In Contributors.kt complement the LoadingStatus enumerate with a LoadingStateData data class. Also, add the INIT state to show an initial status message with only the text: loading status: init. Each time we need to set a new status, we should build an instance of LoadingStatedata class, with the appropriate arguments.

```
enum class LoadingStatus { INIT, COMPLETED, CANCELED, IN_PROGRESS }

data class LoadingStateData(
    val status: LoadingStatus = LoadingStatus.INIT,
    val startTime: Long? = null,
    val elapsedTime: String = ""
)
```

2.3.2 Task 2: Add StateFlow and MutableStateFlow

The immutable StateFlow property in the Contributors interface (in Contributors.kt):

```
val loadingState: StateFlow<LoadingStateData>
```

The Backing Property Pattern for Mutable State Flow in ContributorsUI (in ContributorsUI.kt):

```
/**
 * Private mutable state flow that contains the current loading state data
 * This is the backing field for the public immutable state flow
 */
private val _loadingState =
    MutableStateFlow(Contributors.LoadingStateData())

/**
 * Public immutable state flow that exposes the current loading state
 * Other components can safely observe this without modifying the state
```

```
 */
override val loadingState: StateFlow<Contributors.LoadingStateData> =
    _loadingState.asStateFlow()
```

In this way, changes to `_loadingState.value` will generate a flow item in: `loadingState: StateFlow<Contributors.LoadingStateData>`.

2.3.3 Task 3: Update methods to use StateFlow in loading status

In ContributorsUI.kt define the methods to change loadingStatus property (and generate flow items) and to collect loading status flow items and update UI.

```
/**
 * Updates the loading state by emitting a new value to the StateFlow.
 * This triggers the UI update through the flow collector.
 *
 * @param newStatus New LoadingStateData to emit
 */
override fun updateLoadingStatus(newStatus: Contributors.LoadingStateData)
    {
     _loadingState.value = newStatus
}

/**
 * Sets up a coroutine to observe the loading state flow and update the UI
     accordingly.
 * This method creates a reactive connection between state changes and UI
     updates.
 */
override fun observeLoadingStatus() {
    launch {
        loadingState.collect { status ->
            // Format the status text based on the current state
            val text = "Loading status: " + when (status.status) {
                Contributors.LoadingStatus.COMPLETED -> "completed in
                    ${status.elapsedTime}"
                Contributors.LoadingStatus.IN_PROGRESS -> "in progress
                    ${status.elapsedTime}"
                Contributors.LoadingStatus.CANCELED -> "canceled"
                Contributors.LoadingStatus.INIT -> "init"
            }
            // Update the UI components
            loadingStatus.text = text
            loadingStatus.icon = if (status.status ==
                Contributors.LoadingStatus.IN_PROGRESS) loadingIcon else
                null
        }
    }
}
```

In Contributors.kt define the auxiliary method calculateElapsedTime and abstract method updateLoadingStatus. Also, implement the updateResults method. Additionally, modify the `clearResults()` and `Job.setUpCancellation()` methods to call `updateLoadingStatus` with the appropriate arguments.

```
private fun calculateElapsedTime(startTime: Long): String {
    val time = System.currentTimeMillis() - startTime
    return "${(time / 1000)}.${time % 1000 / 100} sec"
```

```
}

fun updateLoadingStatus(newStatus: LoadingStateData)

private fun updateResults(
    users: List<User>,
    startTime: Long,
    completed: Boolean = true
) {
    updateContributors(users)
    val status = if (completed) COMPLETED else IN_PROGRESS
    val elapsedTime = calculateElapsedTime(startTime)
    updateLoadingStatus(LoadingStateData(status = status, startTime =
        startTime, elapsedTime = elapsedTime))
    if (completed) {
        setActionsStatus(newLoadingEnabled = true)
    }
}
```

2.4 Step 3: DIY, a more robust way to progress updates

Execute this step over Step 2 code. These changes are almost independent of Step 2.

Change Progress Updates to use Channels

From the Contributors.kt - loadContributors() - CHANNELS perspective: instead of calling updateResult after each received item from the channel to get contributors' data, after aggregating the received data, it is required to send data through another channel. This second channel will control the flow of data to the UI, and if it can keep up progress updates, the channel may buffer, suspend, or ignore updates.
This will make progress report management more robust with:

- better flow control - Channels provide back-pressure handling out of the box
- easier testing - Progress updates can be tested independently of the UI
- cleaner cancellation - When the parent coroutine is cancelled, the channel is automatically closed
- Better Error Handling - Errors in processing can be handled in a more structured way
- Separation of Concerns - The producer (progress updates) and consumer (UI updates) are clearly separated

In this new feature, there is only one step to be done:

1. Change Contributors.kt - loadContributors() - CHANNELS to use channels between received requests and the call to updateRequest

2.4.1 Using channels when calling updateRequest

In Contributors.kt - loadContributors() - CHANNELS section, replace existing code with the following one and fix missing code.

```
CHANNELS -> {  // Performing requests concurrently and showing progress

//  launch(Dispatchers.Default) {
//      loadContributorsChannels(service, req) { users, completed ->
//          withContext(Dispatchers.Main) {
//              updateResults(users, startTime, completed)
//          }
//      }
//  }.setUpCancellation()

    launch(Dispatchers.Default) {
        val progressChannel = Channel<Pair<List<User>, Boolean>>(...)

        launch(Dispatchers.Default) {
            loadContributorsChannels(service, req) { users, completed ->
                progressChannel.send(...)
            }
        }

        for (... in progressChannel) {
            withContext(Dispatchers.Main) {
                updateResults(users, startTime, completed)
            }
        }
    }.setUpCancellation()
```

Close channels use to set the end of processing as sooner as possible,

3 Accessing AI LLMs

In AISimpleCall.zip there is an IntelliJ project with a Kotlin console application that makes simple calls to the LLM API of OpenAI and Google Gemini - we call it the AI Assistant.
The AI Assistant implements a polymorphic interface architecture that interacts with OpenAI/Gemini LLMs through RESTful APIs. The system employs JSON based access and typed data classes for request/response serialization, implements configurable temperature-based response modulation (0.1-0.9), and features exponential backoff retry logic. Core functionality is abstracted through a provider-agnostic interface, utilizing factory pattern instantiation. Key technical features include JSON serialization via Gson, HTTP client abstraction with OkHttp, and structured exception handling with coroutine support.
To read information on accessing OpenAI ChatGPT and Google Gemini LLMs, see:

- https://platform.openai.com/docs
- https://ai.google.dev/gemini-api/docs

To access OpenAI ChatGPT or Google Gemini you need an API KEY (for each one of them).
To create an API key, read:

- https://platform.openai.com/account/api-keys
- https://ai.google.dev/gemini-api/docs/api-key

3.1 Task 1: Put the AISimpleCall project to work and test it

After you have a key to access both OpenAI ChatGPT and Google Gemini (or just one of them), add the keys to config.properties and run the code.
Get familiar with the code in the following classes:

- AIAssistant
- AIAssistantFactory
- AIAssistantGemini
- AIAssistantOpenAI
- Main
- Utils
- logback.xml
- config.properties
- README.md

Make tests using the LLMs available, with configuration-property AI_LLM set to OPENAI or GEMINI and check the models that you can access. These configurations access the API using JSON built objects.

After that, get familiar with the code in the following classes:

- AIAssistantGeminiClasses
- AIAssistantOpenAIClasses

Make tests using the LLMs available, with the configuration-property AI_LLM set to OPENAI-CLASSES or GEMINI-CLASSES. These configurations access the API using data classes that are serializable to JSON objects.

3.2 Task 2: Temperature and max_tokens defined as properties

Make the necessary changes to allow setting the *temperature* and *max_tokens* values in the *config.properties* file. Note that these values might be undefined.

3.3 Task 3: Temperature tests

Provide two test cases that demonstrate how changes in the temperature value lead to noticeably different outputs.

3.4 Task 4: Sentiment analysis

Implement changes in the interface to allow for simple input processing **or** sentiment analysis. The sentiment should be evaluated, on a 7-point scale as follows:

1. Very Negative
2. Negative
3. Slightly Negative
4. Neutral
5. Slightly Positive
6. Positive
7. Very Positive

The answer should be the rating number and a justification, in the JSON format:

```
{
  "rating": value,
  "justification": value
}
```

3.5 Task 5: Android LLM Imagem processing

Create a Gemini API Starter project and test it.
Create the project in Android Studio - File - New - New Project... - Gemini API Starter, and then, set your API key.

This Android APP shows several (three) images, with cakes or cookies, and allows the user to send a request to the Gemini AI system with a prompt and the selected image. In this way, we may request the recipe or the suggestion for a name, for the selected cake or cookie, or ask something else.
15% from de 30% extra points - do something extra, in code, in this app.

4 Firebase

4.1 Firebase very short introduction

What is Firebase?
Definition: is a Backend-as-a-Service (BaaS) provided by Google.
What it means: It provides simplified back-end development for mobile and web apps.
What if offers: provides: real-time databases, user authentication, cloud storage, etc.

Firebase core services:

- Authentication - makes authentication easy for end users and developers

- Realtime Database - The Firebase Realtime Database is a cloud-hosted database, where data is stored as JSON and synchronized in realtime to every connected client
- Cloud Firestore - a flexible, scalable database for mobile, web, and server development from Firebase and Google Cloud
- Cloud - a database to store and serve user-generated content, such as photos or videos
- Cloud Functions - serverless framework that lets you automatically run backend code in response to events triggered by Firebase features and HTTPS requests.
- Cloud Messaging - a cross-platform messaging solution that reliably sends messages
- Analytics - app measurement solution, that provides insight on app usage and user engagement
- Crashlytics - a crash reporting solution
- Hosting - App Hosting orchestrates a set of Google Cloud products to deploy, serve, and monitor your web app.

More information about the Firebase ecosystem on: https://firebase.google.com/docs/build

4.2 Codelab Friendly chat

Here, you should execute the codelab: Firebase Android Codelab - Build Friendly Chat.
To help, we provide the code for that codelab in file: *build-android-start.zip*.
The code is complete with the exception of the *google-services.json* file.

First, you should unzip the file into your Android code directory and import the project from the Android Studio (Menu ->File ->New ->Import Project...
The project is a XML Views project and uses the following Firebase services:

- Firebase **Authentication** - to support user authentication
- Firebase **RealTime database** - to store user messages
- **Cloud Storage** - to store binary files (the images).

Then you should follow the codelab, in the link already presented, but you should skip the almost all parts, as the code is already complete. You should read the parts:

1. Overview
6. Enable Authentication
7. Read messages
8. Send Messages

9. Congratulations!
10. Create and set up a Firebase project.

You may read all parts to know what is done, or what could be done (as we will not use Firebase CLI (Command Line Interface)).
In 10 (Optional: Create and set up a Firebase project that you should read and execute:

- Create a Firebase project,
- Upgrade your Firebase pricing plan,
- Add Firebase to your Android project (skip the first two paragraphs, as we will not generate an App key),
- Configure Firebase Authentication (select only Email/Password),
- Set up Realtime Database, and
- Set up Cloud Storage for Firebase (the last topic: Connecting to Firebase resources is not necessary as that code is already removed).

You need to get and put the file *google-services.json* in the *app* folder of your project.

Due to an existing visual bug in the GUI code, in the login screen, the ActionBar overlaps the TextEdit Email. As when the user is already logged in, it jumps straight to MainActivity, we suggest that you start the tests by having a NoActionBar theme, and only after you log in the user, you turn back to an ActionBar theme. You can do that in the SignInActivity.kt code, inside the onStart() method, replacing the theme used:

```
.setTheme(R.style.AppTheme) // or R.style.AppThemeNoActionBar
```

The project presents the Email/Password and Sign in with Google, but the latter is not configured.

4.3 Notes Pro tutorial

Here, you should execute the codelab: Notes App With Firebase — Android — 2024.
To help, we provide the code for that codelab in file: *NotesProXMLViews3.zip*.
The code is complete with the exception of the *google-services.json* file.

First, you should unzip the file into your Android code directory and import the project from the Android Studio (Menu ->File ->New ->Import Project...

The project is an XML Views project and uses the following Firebase services:

- Firebase **Authentication** - to support user authentication
- **Cloud Firestore** - to store the notes.

This project offers an app, in Java code, to store user notes. The app uses email/password with email confirmation, so you need to use real emails that you have account access to: In that way, you may click on the link and validate the email. if you don't do that, the user account will not be activated. You may use the provided code in Kotlin, and you should continue to finish the tutorial from the point where the code is, and you should finish it in Kotlin. However, you should watch the tutorial to synchronize, as you need to get and put the file *google-services.json* in the *app* folder of your project.

The names used are:

- code folder: NoteProXmMLViews3, and
- package path: com.notes.notesproxmlviews.

You are free to start the tutorial from the beginning and choose the names you want.

To get 15% from the 30% extra points, add to this app the possibility of having an optional image in each note..